

$$T(n) = \begin{cases} \textcircled{4} \cdot 1 & n \leq 1 \\ T(m) + T(n-m-1) + \textcircled{4} \cdot n & n > 1 \end{cases}$$

$0 \leq m \leq n-1$

$$m=0$$

$$m=n-1$$

$$T(n) = \textcircled{4} \cdot (n^2)$$

Caso Peggiora $O(n^2)$

Lo dimostriamo per
sostituzione

$$\exists) \exists \bar{n} > 0 \exists c > 0 \forall n \geq \bar{n}$$

$$T(n) \leq c \cdot n^2$$

Per induzione su n

BASE $\bar{n} = 1$

$$T(1) = 2 \underset{\exists}{\leq} c \cdot 1^2 \leadsto \text{vero } \boxed{c \geq 2}$$

PASSO

$$H_{p(Ind)} T(k) \leq c \cdot k^2 \text{ se } k < n \Leftarrow$$

$$\exists) T(n) \leq c \cdot n^2$$

$$T(n) = \max_{0 \leq m \leq n-1} \{ T(m) + T(n-(m+1)) + b \cdot n \}$$

$$\leq \max_{0 \leq m \leq n-1} \{ c \cdot m^2 + c(n-(m+1))^2 + b \cdot n \}$$

$\swarrow$
 Hypothesis
 $m < n$
 $n-(m+1) < n$

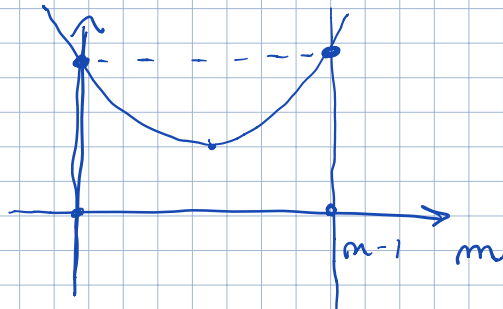
$$\leq \max_{0 \leq m \leq n-1} \{ c \cdot m^2 + c(n-(m+1))^2 + c \cdot n \}$$

$\swarrow$
 $c \geq b$

$$\leq c \cdot \max_{0 \leq m \leq n-1} \{ m^2 + n^2 + (m+1)^2 - 2m(m+1) + n \}$$

$$\leq c \cdot \max_{0 \leq m \leq n-1} \{ \cancel{m^2} + \cancel{m^2} + \cancel{m^2} + \cancel{2m} + 1 - \cancel{2m \cdot m} - \cancel{2m} + n \}$$

$$\leq c \cdot \max_{0 \leq m \leq n-1} \{ 2m^2 - 2m(n-1) + n^2 - n + 1 \} \leq$$



$$m=0 \quad n^2 - n + 1$$

$$m=n-1 \quad 2(n-1)^2 - 2(n-1)^2 + n^2 - n + 1 = n^2 - n + 1$$

$$\leq c \cdot (n^2 - n + 1) \stackrel{F}{\leq} c n^2$$

$$-n + 1 \leq 0 \quad n \geq 1 \quad \checkmark$$

QuickSort nel caso peggiore ha complessità
 $\Theta(n^2)$

Complessità nel caso medio

$$0 \leq m \leq n-1$$

$\sum$

Assumo che tutti i valori di m siano
 equiprobabili

m	0	1	2	...	$n-1$
	$\sum$	$\sum$			$\sum$
Prob.	$\frac{1}{n}$	$\frac{1}{n}$			$\frac{1}{n}$

$$\begin{aligned}
 T_H(n) &= \sum_{m=0}^{n-1} \frac{1}{n} (T_H(m) + T_H(n-(m+1)) + c \cdot n) \\
 &= \frac{1}{n} \sum_{m=0}^{n-1} T_H(m) + T_H(n-(m+1)) + \frac{1}{n} \sum_{m=0}^{n-1} c \cdot n
 \end{aligned}$$

$$\begin{aligned}
&= \frac{1}{n} \left(\underbrace{T_H(0) + T_H(n-1)}_{m=0} + \underbrace{T_H(1) + T_H(n-2)}_{m=1} + \dots \right. \\
&\quad \left. \dots + \underbrace{T_H(n-1) + T_H(0)}_{m=n-1} \right) + \frac{1}{n} \cancel{a} \cdot b \cdot n \\
&= \frac{1}{n} \sum_{m=0}^{n-1} 2 T_H(m) + b \cdot n \\
&= \frac{2}{n} \sum_{m=0}^{n-1} T_H(m) + b \cdot n
\end{aligned}$$

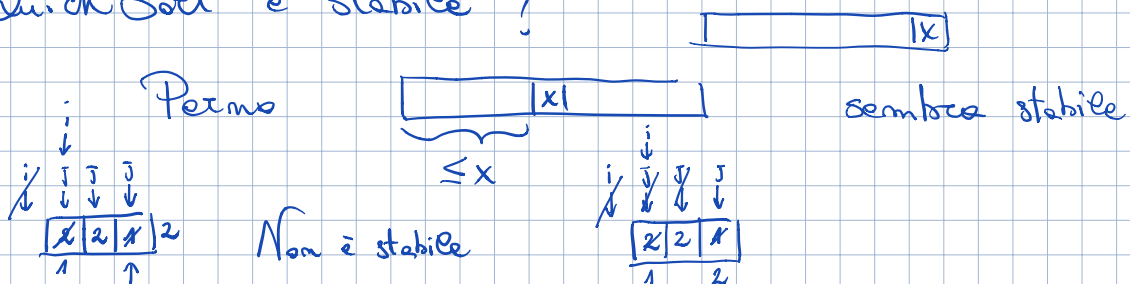
Si dimostra per induzione su n che

$$T_H(n) \leq c \cdot n \log n$$

$$T_H(n) = O(n \log n)$$

QuickSort è in-place? Non è in-place
Ricorsivo

QuickSort è stabile?



Domande

1) È possibile definire un algoritmo di ordinamento che nel caso peggiore sia $O(n \log n)$?

Oppure è vero che sono tutti $\Theta(n \log n)$ nel caso peggiore?

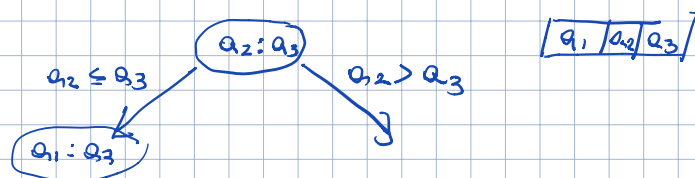
2) Posso migliorare QuickSort e renderlo $\Theta(n \log n)$ (sempre)?
Posso sfruttare meglio Partition?

1) Limitazione Inferiore alla Comp8
complessità dell'ordinamento

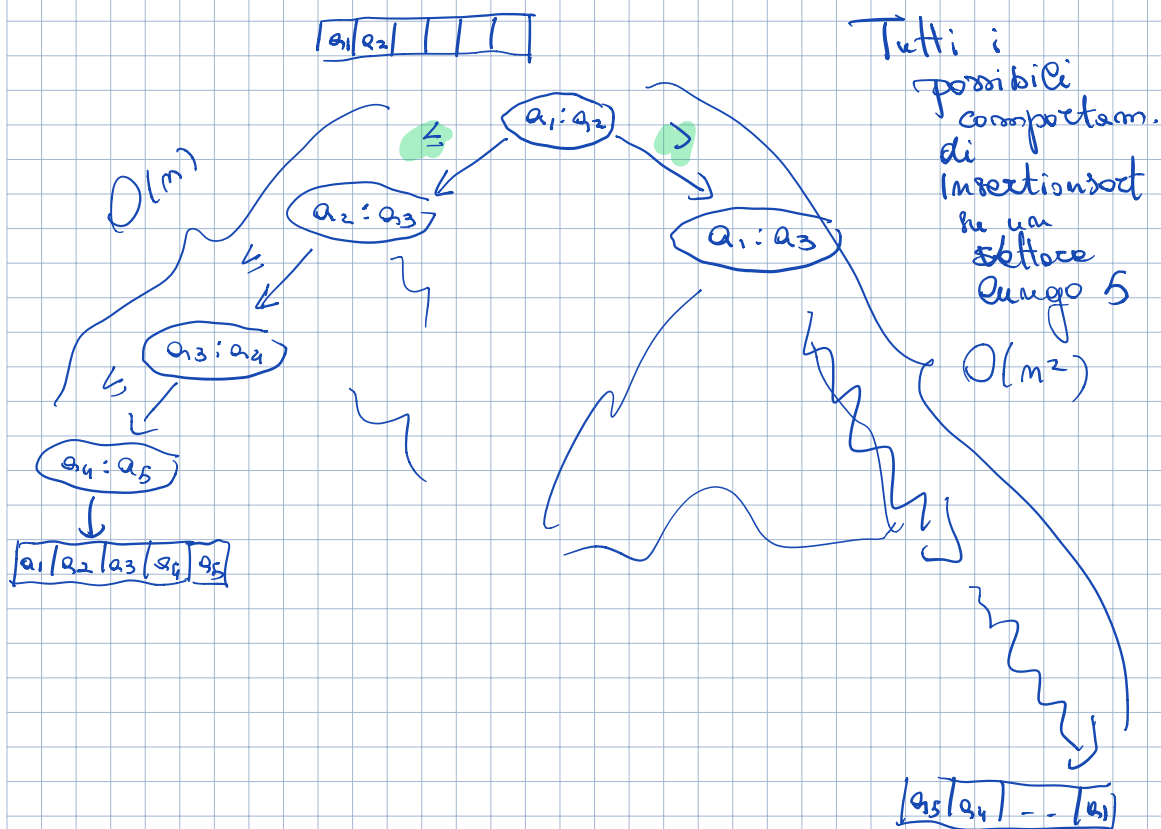
Nessuna ipotesi sui vettori in input

⚡ Operazioni di base Confronti e Scambi

⌋
Alberi delle decisioni



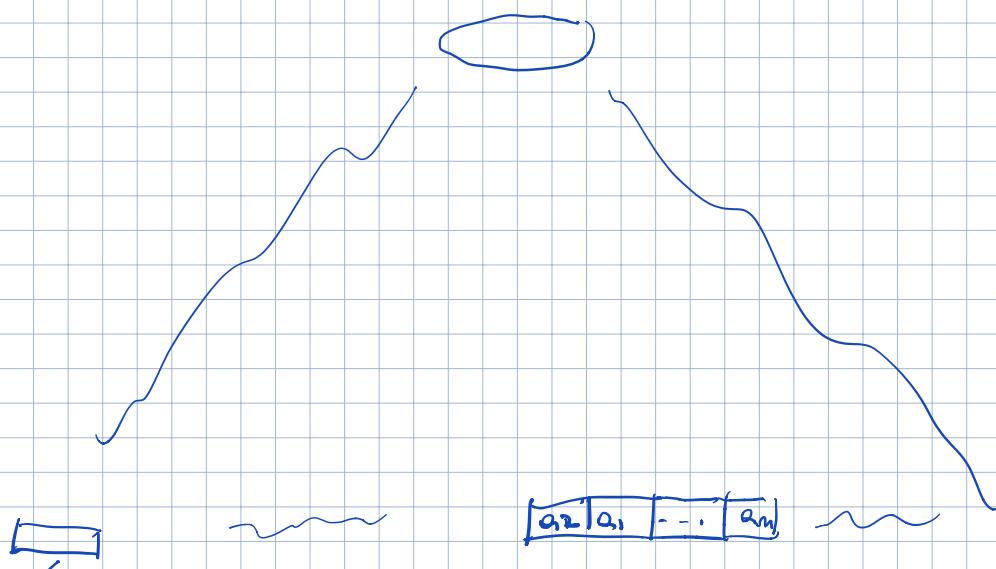
Insertion Sort



Tutti i possibili comportam. di Insertion Sort su un vettore lungo 5

~ . ~ . ~ .

Albero Decisioni Sort su A di lunghezza



3
foglie sono
tutti i possibili
ordin.

$n!$ foglie

Costo di Sort nel caso peggiore?

lung. del cammino più lungo radice - foglie

$\lesssim$
Altezza dell'albero

Quale è il valore minimo dell'altezza
di un albero binario con $n!$ foglie?

$$\Theta(\log n!) = \Theta(n \log n)$$

Tutti gli algoritmi di ordinamento basati
su scambi e confronti nel caso peggiore hanno
complessità $\Omega(n \log n)$.

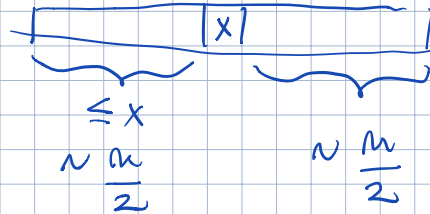
— . — . — .
2) Partition

Problema di QuickSort w/ Pivoto

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + \underbrace{\Theta(n)}_{\Theta(n \log n)} & n \geq 1 \end{cases}$$

Possiamo usare tempo al più $\Theta(n)$ per scegliere
il pivoto

Vorrei in $O(n)$ essere in grado di trovare x in A t.c.



Vorrei in $O(n)$ trovare l'elemento x che finirebbe in posizione $\frac{n}{2}$ se ordinassi A

↕ Generalizzazione

Dato A vettore di lung. n e dato un indice $i \in [1..n]$ determinare l'elemento che finirebbe in posizione i se ordinassi A

In $O(n)$

Selection

Casi particolari

$i=1 \leadsto$ Trova il min $\leadsto$ ok $O(n)$

$i=n \leadsto$ max $''$

$i=5 \leadsto$ Trova il min per 5 volte $\leadsto O(n)$

~~$i = \frac{n}{1000} \leadsto$ Trova il min per $\frac{n}{1000}$ volte $\leadsto O(n^2)$~~

$\leadsto$ HeapSort e rest. $A[\frac{n}{1000}] \leadsto O(n \log n)$